Kanban Board

Project Documentation

Design, implementation and code walkthrough

Project type	Single-page web application (task board)
Technology	HTML5, CSS3, vanilla JavaScript - no frameworks or libraries
Key browser APIs	HTML Drag and Drop API, Web Storage (localStorage), DOM API
Files	index.html (47 lines), style.css (194 lines), script.js (124 lines)
Data storage	Browser only (localStorage, key: tasks)

Contents

Contents 2

1. Introduction and Overview 3

2. Project Structure 4

3. Architecture and Data Flow 5

4. How the Project Was Built 6

5. HTML in Detail 7

6. CSS in Detail 9

7. JavaScript in Detail 10

8. Known Limitations and Recommended Improvements 15

9. Running, Customising and Testing 17

10. Glossary 18

Appendix A - Complete Source Code 19

1. Introduction and Overview

A Kanban board is a visual way to manage work. Tasks are written on cards and the cards move across columns that represent stages of progress. This project is a small Kanban board that runs entirely in the browser. It has three columns - **To Do**, **In progress** and **Done** - and lets the user create task cards, drag them between columns, delete them, and see a live count for each column. Everything is saved in the browser, so the board looks the same after the page is refreshed or the browser is closed.

1.1 Features

Feature	What the user sees	Main code involved
Add task	The *Add new Task* button opens a pop-up with a title and a description. Pressing *Add Task* puts a new card in To Do.	`addTask()`, modal click handlers
Drag and drop	A card can be grabbed and dropped into any column. The target column gets a dashed outline and grows slightly while a card is over it.	drag event listeners, `.hover-over` CSS
Delete task	Each card has a red Delete button that removes the card.	click handler created inside `addTask()`
Live counters	The number next to each column heading always matches the cards in that column.	`updateTaskCount()`
Persistence	Cards survive a page reload.	`localStorage`, JSON, restore block
Modal dialog	A blurred overlay with a form. Clicking outside the form closes it.	`.modal` CSS and JS toggling `.active`

1.2 Technology stack and why it was chosen

Technology	Used for	Reason
HTML5	Structure: navigation bar, three columns, modal form	Semantic tags (nav, main, section) and the `draggable` attribute give structure and drag support without extra tools.
CSS3 (Flexbox, custom properties)	Dark theme, layout, cards, modal, hover animation	Flexbox gives equal-height columns easily; custom properties keep colours and spacing consistent.
Vanilla JavaScript (ES6)	All behaviour	The project is small, so a framework would add more weight than value. Also good for learning how the DOM really works.
HTML Drag and Drop API	Moving cards between columns	Built into browsers, no library needed.
Web Storage (localStorage)	Saving the board	Simple synchronous key/value storage; no server or database needed.

2. Project Structure

```
kanban-board/
 |-- index.html    page markup: navbar, 3 columns, add-task modal
 |-- style.css     theme variables, layout, cards, drag highlight, modal
 |-- script.js     state, add / delete / drag-drop logic, persistence
```

The three files are linked in the usual way: `<link rel="stylesheet" href="style.css">` in the head and `<script src="script.js">` at the very end of the body. Placing the script last matters, because the code selects elements as soon as it runs, so those elements must already exist in the page.

File	Role	Depends on
index.html	Defines what exists on the page. Contains ids and classes that act as hooks for CSS and JS.	style.css, script.js
style.css	Defines how things look: colours, spacing, layout, hover and modal states.	Class names defined in index.html and added by script.js (task, hover-over, active)
script.js	Defines how things behave: creating cards, dragging, counting, saving.	Ids and classes from index.html (todo, progress, done, toggle-modal, modal, ...)

2.1 Page layout

The wireframe below shows the finished interface. The top bar holds the title and the Add button. Below it, three equal columns fill the rest of the window height. Each column has a heading row (name on the left, count on the right) followed by its task cards.

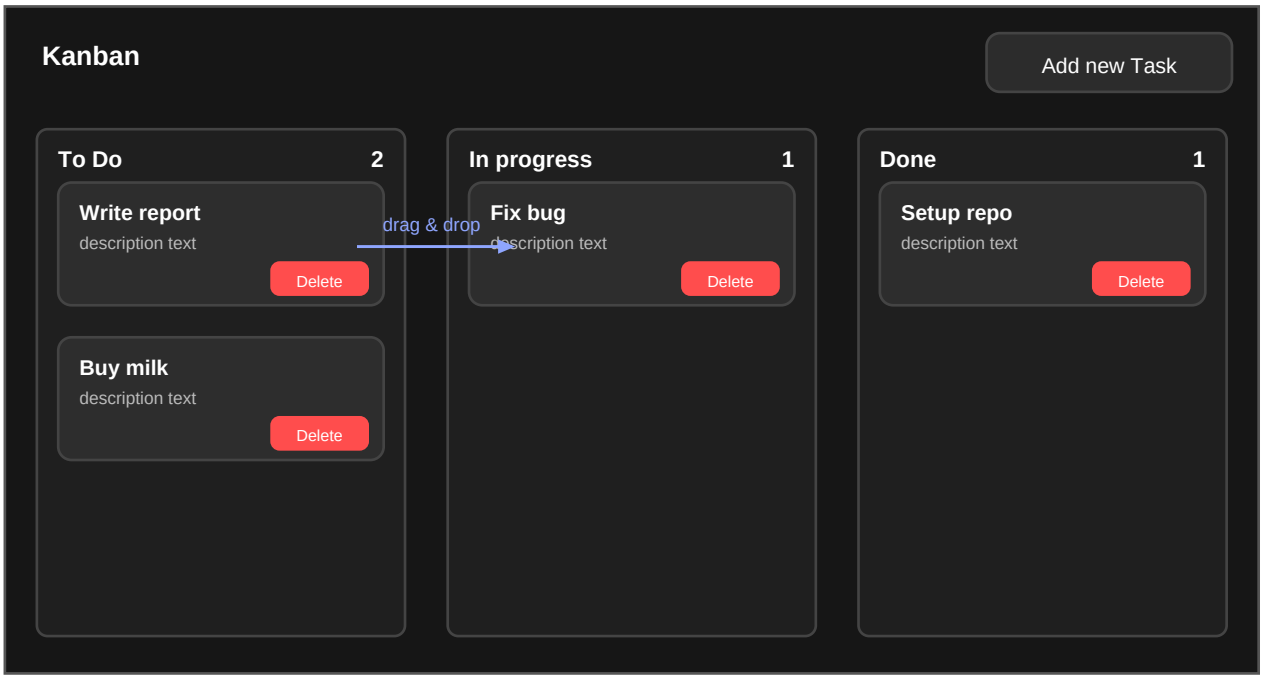


Figure 1 - Layout of the board (dark theme)

3. Architecture and Data Flow

The program has three places where information lives, and most of the code exists to keep them in agreement:

Layer	What it holds	Lifetime
The DOM (task cards on screen)	What the user actually sees. This is the **source of truth** in this project.	Until the page is closed or refreshed
`tasksData` (JavaScript object)	A plain-data snapshot of the DOM: title and description of every card, grouped by column id.	Until the page is closed or refreshed
`localStorage` (key `tasks`)	The same snapshot converted to a JSON string.	Permanent in that browser until cleared

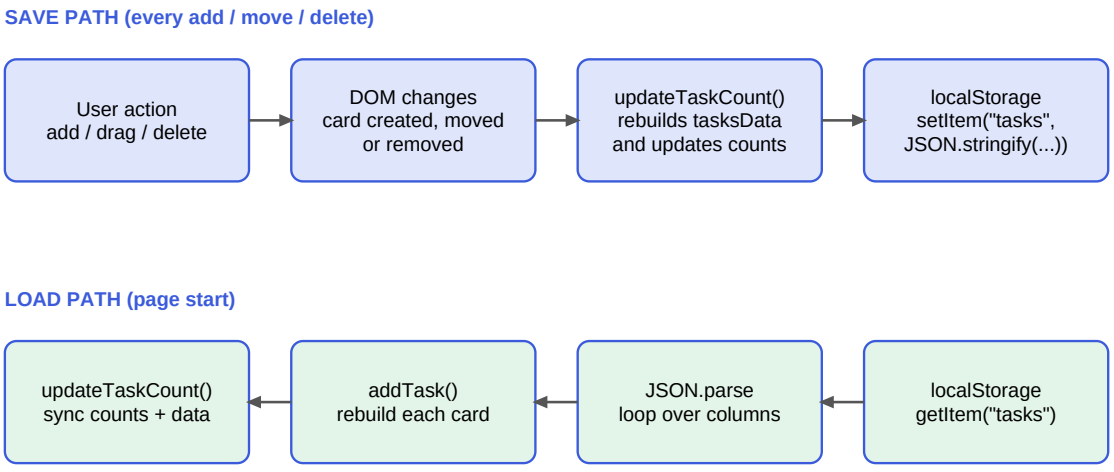
3.1 Data model

Columns are identified by the ids of their elements in the HTML. The saved data is one object whose keys are those ids and whose values are arrays of cards. A card is only its title and description - the column is implied by which array it is in, so moving a card never changes the card itself.

```
{
  "todo":     [ { "title": "Write report", "desc": "Chapter 3 draft" },
               { "title": "Buy milk",     "desc": "2 litres" } ],
  "progress": [ { "title": "Fix bug",      "desc": "Login page" } ],
  "done":     [ { "title": "Setup repo",   "desc": "Git + folders" } ]
}
```

3.2 Data flow

There are two paths. On the **save path**, any user action changes the DOM, then `updateTaskCount()` rebuilds `tasksData` from the DOM and refreshes the counters, and finally the object is written to localStorage. On the **load path**, which runs once when the page opens, the stored JSON is parsed and each card is rebuilt with `addTask()`.



The DOM is the source of truth; tasksData is rebuilt from it before every save.

Figure 2 - Save path and load path

4. How the Project Was Built

The project follows the common order for small front-end apps: structure, then appearance, then behaviour, adding one feature at a time and testing it in the browser before moving on. The steps below describe that process, as it can be reconstructed from the final code.

Step 1 - Plan the features

Before writing code, the feature list was decided: a navbar with an Add button, three columns, a pop-up form, cards with a delete button, drag and drop between columns, per-column counts, and saving. From that list, the data (title, description, column) and the hooks needed (ids for the three columns, a button id, input ids) were identified.

Step 2 - Write the HTML skeleton

- A `<main>` containing a `<nav>` and a `<section class="board">`.
- Three `div.task-column` elements with ids `todo`, `progress`, `done`. Each starts with a `.heading` row holding the name and a count placeholder.
- The modal placed *outside* `<main>` so it can cover the whole window, containing a background layer `.bg` and a form panel `.center`.
- Task cards are intentionally absent from the HTML - JavaScript creates them.

Step 3 - Style the page

1. Reset default margins and set `box-sizing: border-box`; make `html` and `body` fill the window.
2. Declare design tokens (colours, paddings, radii) as CSS variables on `:root`.
3. Lay out the page: `main` as a vertical flex container, `nav` as a space-between row, `section` as a horizontal flex row that grows to fill the leftover height.
4. Style the columns, headings and cards, including the red delete button.
5. Add the `.hover-over` state (dashed border and slight scale-up) used while dragging.
6. Build the modal: hidden by default, shown with the `.active` class, with a blurred translucent background.

Step 4 - Add the JavaScript, feature by feature

1. Select the three columns and put them in a `columns` array.
2. Write `addTask()` to create a card element and place it in a column.
3. Write `updateTaskCount()` to refresh counters and rebuild `tasksData`.
4. Attach drag events to every column so cards can be moved.
5. Wire up the modal: open button, click-outside to close, Add Task button.
6. Add persistence: save to localStorage after each change and restore at start-up.

Step 5 - Test

Each feature was checked manually: adding several tasks, dragging between all column pairs, deleting from each column, and refreshing the page to confirm that cards and counts came back. A checklist is provided in Section 9.

5. HTML in Detail

Full markup (abridged to the structural elements):

```
<main>
  <nav>
    <div class="left">Kanban</div>
    <div class="right"><button id="toggle-modal">Add new Task</button></div>
  </nav>
  <section class="board">
    <div id="todo" class="task-column">
      <div class="heading"><div class="left">To Do</div>
                           <div class="right">Count</div></div>
    </div>
    <div id="progress" class="task-column"> ... In progress ... </div>
    <div id="done" class="task-column"> ... Done ... </div>
  </section>
</main>

<div class="modal add-new-task">
  <div class="bg"></div>
  <div class="center">
    <input type="text" placeholder="Task Title" id="task-title-input"/>
    <textarea placeholder="Task Description" id="task-description-input"></textarea>
    <button id="add-new-task" class="add-task-btn">Add Task</button>
  </div>
</div>
```

5.1 Elements and their purpose

Element	Purpose	Used by
`#toggle-modal`	Button in the navbar that opens (and toggles) the modal.	JS click handler
`#todo, #progress, #done`	The three columns. Their ids are also the keys in the saved data.	JS (selection, drop target, save keys)
`.task-column`	Shared class for all columns.	CSS layout; JS queries tasks inside it
`.heading`	Row holding column name and count.	CSS flex row
`.heading .right`	Displays the number of cards. Starts as the text "Count" and is replaced by JS.	`updateTaskCount()`
`.modal`	Full-window container for the add-task dialog. Hidden unless it has the `active` class.	CSS display rule; JS `classList`
`.modal .bg`	Translucent blurred layer behind the form; clicking it closes the dialog.	JS click handler; CSS
`.modal .center`	The white-box form panel in the middle.	CSS
`#task-title-input, #task-description-input`	Form fields for the new task.	JS reads their `.value`
`#add-new-task`	Button that creates the task.	JS click handler

Note: the modal element has the *class* `add-new-task` and the button has the *id* `add-new-task`. This is valid HTML and the code works, but the shared name can confuse readers; the script only uses the id for the button.

6. CSS in Detail

6.1 Reset and base

The universal selector removes browser default spacing and switches every element to `box-sizing: border-box`, so padding and borders are included in an element's width and height. The `html` and `body` elements are given `height: 100%` so that child layouts can fill the window. The font is the system UI font stack, so the app uses each platform's native look.

6.2 Design tokens (CSS custom properties)

All colours, spacing sizes and corner radii are declared once on `:root` and reused with `var(--name)`. Changing the theme only requires editing these values.

Group	Variables	Value(s)
Colours	`--background-color`, `--background-button-color`, `--background-task-color`, `--primary-text-color`, `--delete-color`	#161616, #1a1a1a, #2a2a2a, #f4f4f4, #ff4d4d
Padding	`--nav-padding`, `--section-padding`, `--padding-s ... --padding-xxl`	15px 30px for nav/section; 5, 10, 15, 20, 30 px scale
Radius	`--border-radius-s ... --border-radius-xxl`, `--border-radius-circle`	5, 10, 15, 20, 30 px and 50%

6.3 Page layout with Flexbox

Rule	Effect
`main { height:100%; display:flex; flex-direction:column; }`	Stacks the navbar above the board and lets the board take the remaining height.
`nav { display:flex; justify-content:space-between; }`	Title on the left, button on the right.
`section { flex-grow:1; display:flex; gap:30px; }`	Board fills the leftover height; columns sit in a row with a 30 px gap.
`.task-column { flex-grow:1; display:flex; flex-direction:column; gap:10px; }`	Columns share the width; inside a column, the heading and cards stack vertically with spacing.

6.4 Cards and delete button

A `.task` is a vertical flex container with a lighter background and rounded corners. Its cursor is `grab` to hint that it can be dragged. The delete button uses `align-self: flex-end` to sit on the right edge of the card and the red `--delete-color` to signal a destructive action.

6.5 Drag highlight

```
section .task-column {
  transition: scale 0.7s cubic-bezier(0.19, 1, 0.22, 1),
              border 0.7s cubic-bezier(0.19, 1, 0.22, 1);
}
section .task-column.hover-over {
  border: 2px dashed var(--primary-text-color);
  scale: 1.05;
}
```

While a card is dragged over a column, JavaScript adds the `hover-over` class. The dashed border and the 5% scale-up show where the card will land, and the transition makes the change smooth. The easing curve `cubic-bezier(0.19, 1, 0.22, 1)` is a fast-start, slow-finish ("ease-out expo") curve that feels snappy. `scale` is the modern standalone CSS property, available in current browsers.

6.6 The modal

Rule	Purpose
`.modal { position:fixed; height:100vh; width:100vw; top:0; left:0; display:none; }`	Covers the whole window and stays in place; hidden by default.
`.modal.active { display:flex; }`	JavaScript adds `active` to show the dialog; flex centres the form.
`.modal .bg { position:absolute; width:100%; height:100%; backdrop-filter:blur(5px); }`	Dimmed, blurred layer that fills the modal and can be clicked to close.
`.modal .center { z-index:10; ... }`	The form panel sits above the background layer so clicks on the form do not close it.
Grouped selector for input, textarea, select, button	Gives all form controls the same dark style with no outline or border.

7. JavaScript in Detail

7.1 Global variables

```
let tasksData = {};

const todo     = document.querySelector("#todo");
const progress = document.querySelector("#progress");
const done     = document.querySelector("#done");

const columns = [todo, progress, done];
let draggedElement = null;
```

Variable	Meaning
`tasksData`	In-memory snapshot of all cards, rebuilt by `updateTaskCount()`. It is what gets saved.
`todo, progress, done`	References to the three column elements, found once with `querySelector`.
`columns`	The same three elements in an array so code can loop over them.

Variable	Meaning
`draggedElement`	The card currently being dragged. Set at the start of a drag and read when it is dropped.

7.2 addTask(title, desc, column)

```
function addTask(title, desc, column) {
  const div = document.createElement("div");
  div.classList.add("task");
  div.setAttribute("draggable", "true");

  div.innerHTML = `
        <h2>${title}</h2>
        <p>${desc}</p>
        <button>Delete</button>
    `;

  column.appendChild(div);

  div.addEventListener("dragstart", () => { draggedElement = div; });

  div.querySelector("button").addEventListener("click", () => {
    div.remove();
    updateTaskCount();
    localStorage.setItem("tasks", JSON.stringify(tasksData));
  });
}
```

This function is the single place where a card is created, both for new tasks and for tasks restored after a reload. It works in six steps:

1 Create an empty `div` and give it the class `task` (CSS styles it).

2 Set `draggable="true"`. Without this attribute the browser will not start a drag on a div.

3 Fill the card using a template literal containing a heading, a paragraph and a Delete button.

4 Append the card to the requested column.

5 Register `dragstart`: when dragging begins, remember which card it is.

6 Register the Delete button: remove the card, recalculate counts/data, and save.

Because the listeners are attached to the card object itself, they travel with it when the card is moved to another column, so a moved card can still be dragged and deleted. Note that `addTask()` does not update counts or save by itself. The caller does that, which lets the restore loop add many cards and then update once.

7.3 updateTaskCount()

```
function updateTaskCount() {
  columns.forEach((col) => {
    const tasks = col.querySelectorAll(".task");
    const count = col.querySelector(".right");

    tasksData[col.id] = Array.from(tasks).map((t) => {
      return {
        title: t.querySelector("h2").innerText,
        desc:  t.querySelector("p").innerText,
      };
    });

    count.innerText = tasks.length;
  });
}
```

Despite its name, this function does two jobs for each column: it **refreshes the counter** and it **rebuilds the saved data**.

- `col.querySelectorAll('.task')` returns all cards currently inside the column.
- `col.querySelector('.right')` finds the first element with class right inside the column - the count label in the heading (cards contain no such element).
- `Array.from(tasks).map(...)` converts the list of card elements into plain objects with title and description, stored in `tasksData` under the column id.
- The count label is set to the number of cards.

Since it reads from the DOM every time, the data can never drift away from what the user sees, no matter how the cards got there.

7.4 Restoring saved tasks

```
if (localStorage.getItem("tasks")) {
  const data = JSON.parse(localStorage.getItem("tasks"));

  for (const col in data) {
    const column = document.querySelector(`#${col}`);

    data[col].forEach((task) => {
      addTask(task.title, task.desc, column);
    });
  }
}

updateTaskCount();
```

1 Check whether anything was saved under the key `tasks`.

2 Convert the JSON string back into an object with `JSON.parse`.

3 Loop over its keys (todo, progress, done). Each key doubles as the id of a column element, so `#${col}` finds the right column.

4 Call `addTask()` for every saved card to rebuild it in its column.

5 Finally call `updateTaskCount()`. This runs even on a first visit, replacing the "Count" placeholder with 0.

7.5 Drag and drop

The HTML Drag and Drop API works through a sequence of events. Cards fire `dragstart`; columns react to `dragenter`, `dragover`, `dragleave` and `drop`.

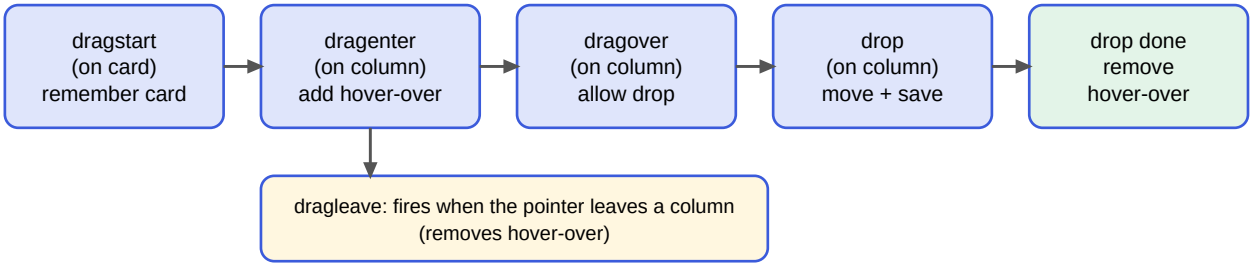


Figure 3 - Drag and drop event sequence

```
function addDragEventsOnColumn(column) {
  column.addEventListener("dragenter", (e) => {
    e.preventDefault();
    column.classList.add("hover-over");
  });
  column.addEventListener("dragleave", (e) => {
    e.preventDefault();
    column.classList.remove("hover-over");
  });
  column.addEventListener("dragover", (e) => { e.preventDefault(); });

  column.addEventListener("drop", (e) => {
    e.preventDefault();
    if (!draggedElement) return;
    column.appendChild(draggedElement);
    column.classList.remove("hover-over");
    updateTaskCount();
    localStorage.setItem("tasks", JSON.stringify(tasksData));
  });
}
addDragEventsOnColumn(todo);  addDragEventsOnColumn(progress);  addDragEventsOnColumn(done);
```

Event	Fires on	What the code does and why
`dragstart`	The card	Stores the card in `draggedElement` so the drop handler knows what to move.
`dragenter`	A column	Adds `hover-over` to highlight the target. `preventDefault()` signals that this element accepts drops.
`dragover`	A column (repeatedly)	`preventDefault()` is required: by default browsers refuse drops, and cancelling this event is how an element declares itself a valid drop zone.
`dragleave`	A column	Removes the highlight when the pointer leaves.
`drop`	A column	Moves the card with `appendChild` (which relocates an existing node rather than copying it), clears the highlight, recounts and saves. The guard `if (!draggedElement) return` ignores drops that did not start from a card, such as a file dragged in from the desktop.

7.6 Modal logic

```
const toggleModalBtn = document.querySelector("#toggle-modal");
const modalbg  = document.querySelector(".modal .bg");
const modal    = document.querySelector(".modal");
const addTaskBtn = document.querySelector("#add-new-task");

toggleModalBtn.addEventListener("click", () => { modal.classList.toggle("active"); });
modalbg.addEventListener("click", () => { modal.classList.remove("active"); });

addTaskBtn.addEventListener("click", () => {
  const taskTitle = document.querySelector("#task-title-input").value;
  const taskDesc  = document.querySelector("#task-description-input").value;

  addTask(taskTitle, taskDesc, todo);
  updateTaskCount();
  localStorage.setItem("tasks", JSON.stringify(tasksData));

  modal.classList.remove("active");
});
```

- **Open:** the navbar button toggles the `active` class. CSS turns `display:none` into `display:flex`, so no show/hide code is needed in JS.
- **Close:** clicking the blurred background removes `active`. Clicks inside the form do not reach the background, because the form is a separate sibling layer.
- **Add:** reads the two input values, creates the card in To Do (new tasks always start there), recounts, saves, and closes the dialog.

7.7 Order of execution when the page loads

1. The browser builds the DOM from index.html and applies style.css.
2. script.js runs last: selects the columns and defines the functions.
3. The restore block rebuilds any saved cards.
4. `updateTaskCount()` fills in the counters.
5. Drag listeners are attached to the three columns.
6. Modal listeners are attached. The board is now interactive.

7.8 User flows

Flow	Sequence
Add a task	Click *Add new Task* → modal gets `.active` → type title and description → click *Add Task* → `addTask()` appends card to To Do → `updateTaskCount()` refreshes counts and data → saved → modal closes.
Move a task	Press on a card (`dragstart` stores it) → move over a column (`dragenter` highlights it) → release (`drop` appends the card there) → counts update → saved.
Delete a task	Click Delete → card removed from the DOM → counts and data rebuilt without it → saved.
Reload the page	Script runs → saved JSON parsed → each card rebuilt in its column → counts updated.

8. Known Limitations and Recommended Improvements

The code works for its purpose, and reading it closely also shows several places where it can be made safer and more polished. They are grouped by importance.

8.1 Important

Issue	Why it matters	Recommended fix
Text is inserted with `innerHTML`	If a title or description contains HTML tags, the browser treats them as markup instead of text. This can break the layout and, in a shared or networked app, is a security risk (cross-site scripting). It also means characters like < may be altered when saved text is read back and re-inserted.	Build the card with `createElement` and set `textContent` for the title and description.
No input validation	Empty titles create blank cards.	Trim the values and refuse to add when the title is empty.
Saving code repeated in four places	Add, delete, drop and modal all repeat the update-and-save pair, so a change must be made four times.	Create one `saveTasks()` function that calls `updateTaskCount()` and writes to localStorage.
Unprotected restore	If the stored value is corrupted, `JSON.parse` throws and the whole script stops. A saved key that matches no element makes `column` null and also throws.	Wrap restore in try/catch and skip unknown column ids.

8.2 Usability and polish

Observation	Suggested improvement
The modal inputs keep the previous text after a task is added.	Clear both fields after adding.
`draggedElement` is never reset after a drop, and there is no `dragend` handler (so a cancelled drag leaves highlights behind).	Add a `dragend` listener that clears `draggedElement` and removes `hover-over` from all columns.
`dragleave` also fires when the pointer moves over a child element inside the column, which can make the highlight flicker.	Check `e.relatedTarget` before removing the class, or use a drag counter.
Cards always drop at the end of a column; they cannot be ordered within it.	On drop, find the card under the pointer and insert before it.
The column has no border until `hover-over` adds 2 px, so content shifts slightly, and borders from none to dashed cannot animate.	Give columns a transparent 2 px border by default.
The HTML Drag and Drop API works poorly on touch screens.	Add touch/pointer-event handling, or a library, for mobile.
No way to edit a task, or close the dialog with the Escape key.	Add an edit mode and a `keydown` handler for Escape.
Columns share width with `flex-grow` only, so very long text can make columns unequal; no media queries for small screens.	Use `flex:1; min-width:0` and add a responsive layout (stack columns on narrow screens).
Each card gets its own listeners.	Use event delegation: one listener on each column or the board.
Drag handles and buttons lack accessibility support.	Add ARIA labels and a keyboard way to move cards (e.g. buttons).

8.3 Example of a safer `addTask()` with a shared save function

```
function saveTasks() {
  updateTaskCount();
  localStorage.setItem("tasks", JSON.stringify(tasksData));
}

function addTask(title, desc, column) {
  const div = document.createElement("div");
  div.classList.add("task");
  div.draggable = true;

  const h2 = document.createElement("h2");   h2.textContent = title;
  const p  = document.createElement("p");    p.textContent  = desc;
  const btn = document.createElement("button"); btn.textContent = "Delete";
  div.append(h2, p, btn);
  column.appendChild(div);

  div.addEventListener("dragstart", () => { draggedElement = div; });
  div.addEventListener("dragend",   () => { draggedElement = null; });
  btn.addEventListener("click", () => { div.remove(); saveTasks(); });
}
```

8.4 Feature ideas

- Edit existing tasks; add due dates, priority labels or colour tags.
- Search/filter and sort inside columns; custom columns.
- Export/import the board as a JSON file.
- Sync across devices with a backend or a cloud database.

9. Running, Customising and Testing

9.1 How to run

1. Put index.html, style.css and script.js in the same folder.
2. Open index.html in a modern browser (Chrome, Edge, Firefox or Safari). No server or build step is needed.
3. Click *Add new Task*, fill in the form and press *Add Task*.
4. Drag cards between columns; click Delete to remove them. Refresh the page to confirm they persist.
5. To reset the board, clear the site's data or run `localStorage.removeItem("tasks")` in the browser console.

9.2 Customisation

To change	Where
Colours and spacing	CSS variables in `:root` at the top of style.css.
Column names	The `.heading .left` text of each column in index.html.
Add a column	Add a new `div.task-column` with a unique id and heading in index.html; add it to the `columns` array, call `addDragEventsOnColumn()` on it, and the rest (counts, saving, restoring) works automatically.
Highlight animation	The `transition` and `.hover-over` rules in style.css.
Storage key	The string `"tasks"` used in script.js (appears several times).

9.3 Manual test checklist

#	Test	Expected result
1	Open the page for the first time	Three empty columns, each showing 0
2	Add a task with title and description	Card appears in To Do; To Do count becomes 1
3	Add three more tasks	Count updates to 4; cards stack in order
4	Drag a card from To Do to In progress	Target highlights while hovering; card moves; both counts update
5	Drag a card from In progress to Done, then back to To Do	Works in every direction
6	Delete a card from each column	Card disappears; that column's count decreases
7	Refresh the page	All remaining cards are in the same columns; counts correct
8	Click outside the form while the dialog is open	Dialog closes; nothing is added
9	Add a task with an empty title	Currently adds an empty card (see 8.1)
10	Type HTML tags such as bold tags in a title	Currently rendered as markup (see 8.1)

10. Glossary

Term	Meaning
DOM	Document Object Model - the browser's live tree of page elements that JavaScript can read and change.
Event listener	A function registered with `addEventListener` that runs when a given event (click, drop, ...) occurs.
localStorage	Browser key/value storage that stores strings and persists after the browser is closed.
JSON	A text format for data. `JSON.stringify` turns an object into JSON text; `JSON.parse` turns it back.
Template literal	A string in backticks that can contain `${expression}` placeholders.
Flexbox	A CSS layout system that arranges items in a row or column and distributes the space.
CSS custom property	A variable like `--delete-color` declared in CSS and used with `var()`.
Drop zone	An element that has told the browser it accepts dragged items, by cancelling the `dragover` event.

Appendix A - Complete Source Code

The three project files are reproduced in full for reference. Line numbers are added for easy cross-reference and are not part of the code.

A.1 index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Kanban Board</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <main>
      <nav>
        <div class="left">Kanban</div>
        <div class="right"><button id="toggle-modal">Add new Task</button></div>
      </nav>
      <section class="board">
        <div id="todo" class="task-column">
          <div class="heading">
            <div class="left">To Do</div>
            <div class="right">Count</div>
          </div>
        </div>
        <div id="progress" class="task-column">
          <div class="heading">
            <div class="left">In progress</div>
            <div class="right">Count</div>
          </div>
        </div>
        <div id="done" class="task-column">
          <div class="heading">
            <div class="left">Done</div>
            <div class="right">Count</div>
          </div>
        </div>
      </section>
    </main>

    <div class="modal add-new-task">
      <div class="bg"></div>
      <div class="center">
        <input type="text" placeholder="Task Title" id="task-title-input"/>
        <textarea placeholder="Task Description" id="task-description-input"></textarea>
        <button id="add-new-task" class="add-task-btn">Add Task</button>
      </div>
    </div>
    <script src="script.js"></script>
  </body>
</html>
```

A.2 script.js

```
let tasksData = {};

const todo = document.querySelector("#todo");
const progress = document.querySelector("#progress");
const done = document.querySelector("#done");

const columns = [todo, progress, done];

let draggedElement = null;

function addTask(title, desc, column) {
  const div = document.createElement("div");
  div.classList.add("task");
  div.setAttribute("draggable", "true");

  div.innerHTML = `
        <h2>${title}</h2>
        <p>${desc}</p>
        <button>Delete</button>
    `;

  column.appendChild(div);

  div.addEventListener("dragstart", () => {
    draggedElement = div;
  });

  // Delete button
  div.querySelector("button").addEventListener("click", () => {
    div.remove();
    updateTaskCount();
    localStorage.setItem("tasks", JSON.stringify(tasksData));
  });
}

function updateTaskCount() {
  columns.forEach((col) => {
    const tasks = col.querySelectorAll(".task");
    const count = col.querySelector(".right");

    tasksData[col.id] = Array.from(tasks).map((t) => {
      return {
        title: t.querySelector("h2").innerText,
        desc: t.querySelector("p").innerText,
      };
    });

    count.innerText = tasks.length;
  });
}

// Restore tasks
if (localStorage.getItem("tasks")) {
  const data = JSON.parse(localStorage.getItem("tasks"));

  for (const col in data) {
    const column = document.querySelector(`#${col}`);

    data[col].forEach((task) => {
      addTask(task.title, task.desc, column);
    });
  }
}

updateTaskCount();

function addDragEventsOnColumn(column) {
  column.addEventListener("dragenter", (e) => {
    e.preventDefault();
    column.classList.add("hover-over");
  });

  column.addEventListener("dragleave", (e) => {
    e.preventDefault();
    column.classList.remove("hover-over");
  });

```

```
  column.addEventListener("dragover", (e) => {
    e.preventDefault();
  });

  column.addEventListener("drop", (e) => {
    e.preventDefault();

    if (!draggedElement) return;

    column.appendChild(draggedElement);
    column.classList.remove("hover-over");

    updateTaskCount();
    localStorage.setItem("tasks", JSON.stringify(tasksData));
  });
}

addDragEventsOnColumn(todo);
addDragEventsOnColumn(progress);
addDragEventsOnColumn(done);

// Modal logic
const toggleModalBtn = document.querySelector("#toggle-modal");
const modalbg = document.querySelector(".modal .bg");
const modal = document.querySelector(".modal");
const addTaskBtn = document.querySelector("#add-new-task");

toggleModalBtn.addEventListener("click", () => {
  modal.classList.toggle("active");
});

modalbg.addEventListener("click", () => {
  modal.classList.remove("active");
});

addTaskBtn.addEventListener("click", () => {
  const taskTitle = document.querySelector("#task-title-input").value;
  const taskDesc = document.querySelector("#task-description-input").value;

  addTask(taskTitle, taskDesc, todo);

  updateTaskCount();
  localStorage.setItem("tasks", JSON.stringify(tasksData));

  modal.classList.remove("active");
});
```

A.3 style.css

```css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html,
body {
  height: 100%;
  width: 100%;
}

html {
  font-family:
    system-ui,
    -apple-system,
    BlinkMacSystemFont,
    "Segoe UI",
    Roboto,
    Oxygen,
    Ubuntu,
    Cantarell,
    "Open Sans",
    "Helvetica Neue",
    sans-serif;
}

:root {
  /* colors */
  --background-color: #161616;
  --background-button-color: #1a1a1a;
  --background-task-color: #2a2a2a;
  --primary-text-color: #f4f4f4;
  --delete-color: #ff4d4d;

  /* padding variables */
  --nav-padding: 15px 30px;
  --section-padding: 15px 30px;
  --padding-s: 5px;
  --padding-m: 10px;
  --padding-l: 15px;
  --padding-xl: 20px;
  --padding-xxl: 30px;

  /* border radius variables */
  --border-radius-s: 5px;
  --border-radius-m: 10px;
  --border-radius-l: 15px;
  --border-radius-xl: 20px;
  --border-radius-xxl: 30px;
  --border-radius-circle: 50%;
}

body {
  background-color: var(--background-color);
  color: var(--primary-text-color);
}

main {
  height: 100%;
  display: flex;
  flex-direction: column;
}

nav {
  padding: var(--nav-padding);
  display: flex;
  align-items: center;
  justify-content: space-between;
}

nav button {
  background-color: var(--background-button-color);
  border: none;
  outline: none;
  color: var(--primary-text-color);
  padding: var(--padding-m) var(--padding-l);
```

```
  cursor: pointer;
  border-radius: var(--border-radius-m);
}

section {
  flex-grow: 1;
  padding: var(--section-padding);
  display: flex;
  justify-content: space-between;
  gap: var(--padding-xxl);
}

section .task-column {
  background-color: var(--background-button-color);
  flex-grow: 1;
  padding: var(--padding-m);
  display: flex;
  flex-direction: column;
  gap: var(--padding-m);
  border-radius: var(--border-radius-s);
   transition:
    scale 0.7s cubic-bezier(0.19, 1, 0.22, 1),
    border 0.7s cubic-bezier(0.19, 1, 0.22, 1);
}

section .task-column.hover-over {
  border: 2px dashed var(--primary-text-color);

  scale: 1.05;
}

section .task-column .heading {
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: space-between;
  transition: all 0.7s cubic-bezier(0.19, 1, 0.22, 1);
}

section .task-column .task {
  background-color: var(--background-task-color);
  padding: var(--padding-m);
  border-radius: var(--border-radius-m);
  display: flex;
  flex-direction: column;
  gap: var(--padding-s);
  cursor: grab;
}

section .task-column .task button {
  align-self: flex-end;
  cursor: pointer;
  padding: var(--padding-s) var(--padding-m);
  border-radius: var(--border-radius-s);
  border: none;
  outline: none;
  background-color: var(--delete-color);
  color: var(--primary-text-color);
}

.modal{
    position: fixed;
    height: 100vh;
    width: 100vw;
    top:0;
    left:0;
    display: none;
    align-items: center;
    justify-content: center;

}

.modal.active{
    display:flex;
}

.modal .bg{
    position: absolute;
    height: 100%;
```

```
    width: 100%;
    background-color: rgba(26, 25, 25, 0.499);
    backdrop-filter: blur(5px);

}

.modal .center {
    background-color: var(--background-button-color);
    padding: var(--padding-xxl);
    border-radius: var(--border-radius-m);
    display: flex;
    flex-direction: column;
    gap: var(--padding-m);
    z-index: 10;
}

.modal .center input,
.modal .center textarea,
.modal .center select {
    background-color: var(--background-task-color);
    border: none;
    outline: none;
    color: var(--primary-text-color);
    padding: var(--padding-m);
    border-radius: var(--border-radius-s);
}

.modal .center button {
    background-color: var(--background-task-color);
    border: none;
    outline: none;
    color: var(--primary-text-color);
    padding: var(--padding-m) var(--padding-l);
    cursor: pointer;
    border-radius: var(--border-radius-m);
    width:fit-content;
    align-self: center;
}
```